Implement simple cache with direct mapping

The assignment is to implement an 8x4 direct mapping cache with 8 cache lines (or blocks) where each line holds a block size of 4 words.

Cache line = Int(Address/4) mod 8

Cache column = Address mod 4

The CSCI6461 Simulator was modified to include a display box showing live cache. It displays the cache tag, 4-word content, and a dirty bit indicating the cache was written into. When loading a value from memory, first the cache is checked. If it's a hit, the value is fetched from cache. If it's a miss, the value is retrieved from memory and cache is updated. On a store hit, meaning the address is in the cache, the cache is updated and the dirty bit is set to 1, but main memory is not updated. This is write-back. On a store miss, the cache is updated from memory, the cache is then updated with new word that needs to be stored, and the dirty bit set to 1. This is write-allocate. If a cache block with the dirty bit set to1 is evicted, it is first written to memory.

The write-back, write-allocate cache handles all program memory accesses, including instruction fetches, LDR/STR, LDX/STX, AMR/SMR, and indirect lookups, providing a single path to memory from the program's perspective. Front panel Load and Store buttons deliberately bypass the cache to access the memory array directly, simulating an operator rather than a program.

Code Changes

In addition to the html code change to add a table, changes were made to the javascript code within executeInstruction(). Instructions that were updated:

LDR r, x, address[,I]	Load Register From Memory
STR r, x, address[,I]	Store Register To Memory
LDX x, address[,I]	Load Index Register from Memory
STX x, address[,I]	Store Index Register to Memory
AMR r, x, address[,I]	Add Memory To Register
SMR r, x, address[,I]	Subtract Memory From Register

The changes involve two javascript functions getWord() and storeWord(). getWord() fetches a word at a specified memory address. The first part checks whether the address is in cache based on cache row and tag which can be computed from the address. If there is a match then get the word from cache. Otherwise it gets the word from memory and updates cache using updateCachefromMemory() which will check if memory needs to be updated before evicting. storeWord() stores a word at the specified memory address. If the address is in cache, it overwrites that word and sets the dirty bit to 1. If the address is not in cache, it updates cache from memory using updateCachefromMemory() which will check if memory needs to be updated before evicting. After cache is updated storeWord is called recursively.

In addition, these functions were updated to use cache instead of memory:

computeEA()

computeIndexEA(

computeLDAEA()

fetchInstruction()

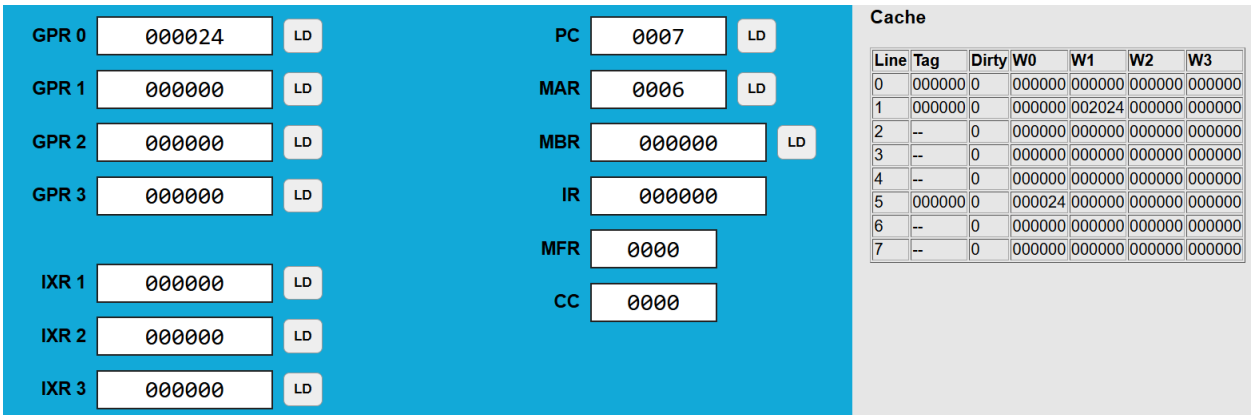
Also, resetMachine() resets cache in addition to its other functionality.

Example of loading (Ex.asm)

Memory	20	21	22	23	...	28	29	30	31	...	52	53	54	55
	20	0	0	0		0	0	0	52		100	0	0	0

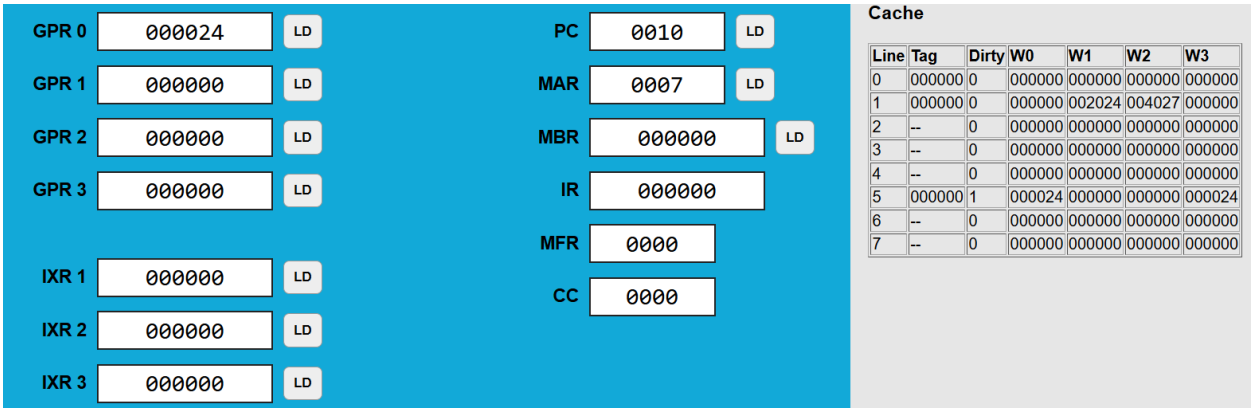
Address	Octal Address	word	command
5	000005	002024	LDR 0, 0, 20
6	000006	004027	STR 0, 0, 23
7	000007	102137	LDX 1, 31
8	000010	003500	LDR 3, 1, 0
9	000011	002427	LDR 1, 0, 23

The first instruction 002024 (LDR 0, 0, 20) is at address 5. It is loaded into the second row of cache. Since it is an LDR command, the block of memory starting at address 20 is loaded into the 6th row of the cache and GPR 0.



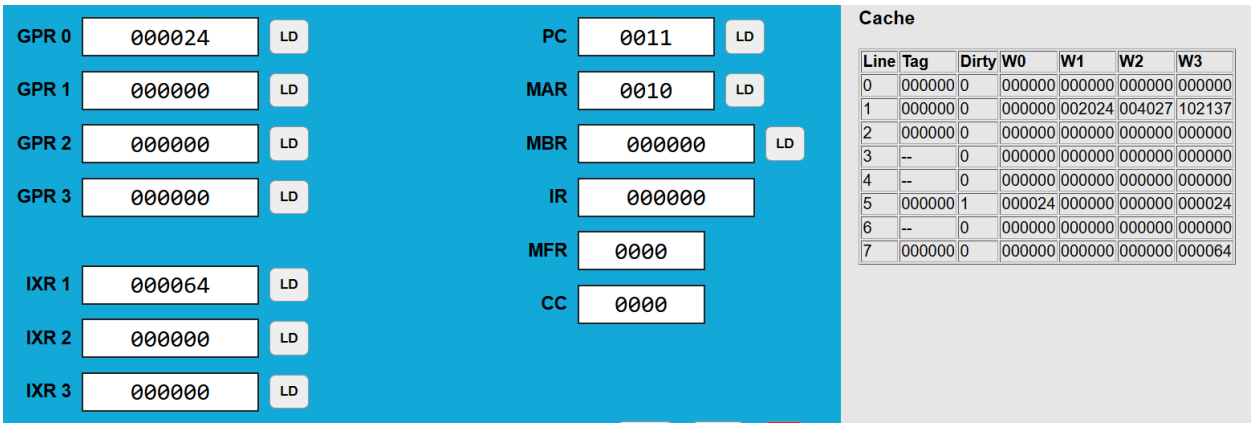
Line	Tag	Dirty	W0	W1	W2	W3
0	000000	0	000000	000000	000000	000000
1	000000	0	000000	002024	000000	000000
2	--	0	000000	000000	000000	000000
3	--	0	000000	000000	000000	000000
4	--	0	000000	000000	000000	000000
5	000000	0	000024	000000	000000	000000
6	--	0	000000	000000	000000	000000
7	--	0	000000	000000	000000	000000

The next instruction 004027 (STR 0, 0, 23) is loaded into the second row of cache. It is a store command for address 23, so 20 is stored in the 6th row and 4th column of cache. The dirty bit is set to 1.



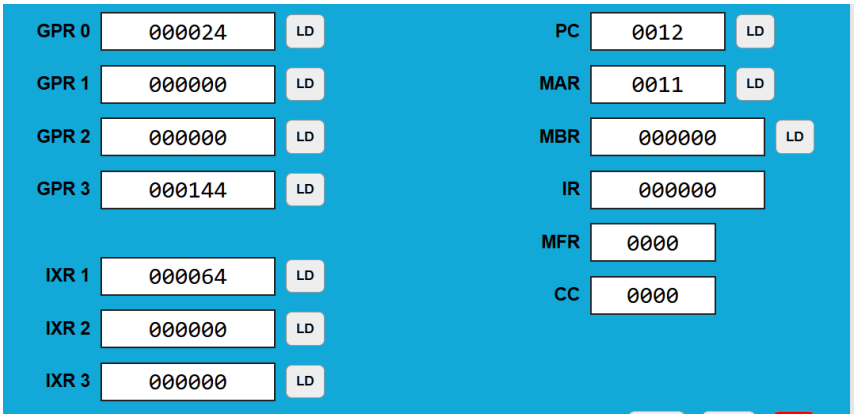
Line	Tag	Dirty	W0	W1	W2	W3
0	000000	0	000000	000000	000000	000000
1	000000	0	000000	002024	004027	000000
2	--	0	000000	000000	000000	000000
3	--	0	000000	000000	000000	000000
4	--	0	000000	000000	000000	000000
5	000000	1	000024	000000	000000	000024
6	--	0	000000	000000	000000	000000
7	--	0	000000	000000	000000	000000

The next instruction 102137 (LDX 1, 31) is loaded into the second row of cache in column 4. It loads the number 52 from address 31 into both cache row 8 and index register 1.



Line	Tag	Dirty	W0	W1	W2	W3
0	000000	0	000000	000000	000000	000000
1	000000	0	000000	002024	004027	102137
2	000000	0	000000	000000	000000	000000
3	--	0	000000	000000	000000	000000
4	--	0	000000	000000	000000	000000
5	000000	1	000024	000000	000000	000024
6	--	0	000000	000000	000000	000000
7	000000	0	000000	000000	000000	000064

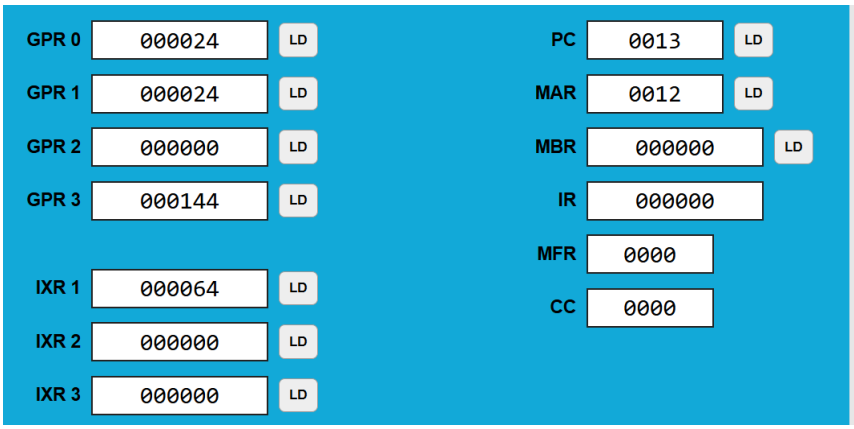
The next instruction 003500 (LDR 3, 1, 0) is loaded into the third row of cache in column 1. It loads the number 100 from address 52 into GPR 3. The block of memory starting at address 52 is then loaded into the 6th row of the cache which also changes the tag. The block of memory starting at address 20 is evicted. Since the dirty bit was 1, it was written to main memory upon eviction.



Cache

Line	Tag	Dirty	W0	W1	W2	W3
0	000000	0	000000	000000	000000	000000
1	000000	0	000000	002024	004027	102137
2	000000	0	003500	000000	000000	000000
3	--	0	000000	000000	000000	000000
4	--	0	000000	000000	000000	000000
5	000001	0	000144	000000	000000	000000
6	--	0	000000	000000	000000	000000
7	000000	0	000000	000000	000000	000064

The final instruction 002427 (LDR 1, 0, 23) is loaded into the third row of cache in column 2. The block of memory starting at address 20 is again loaded into the 6th row of the cache after loading GPR 1 with the contents of memory address 23. The value is now 20 instead of 0, showing main memory was updated correctly.



Cache

Line	Tag	Dirty	W0	W1	W2	W3
0	000000	0	000000	000000	000000	000000
1	000000	0	000000	002024	004027	102137
2	000000	0	003500	002427	000000	000000
3	--	0	000000	000000	000000	000000
4	--	0	000000	000000	000000	000000
5	000000	0	000024	000000	000000	000024
6	--	0	000000	000000	000000	000000
7	000000	0	000000	000000	000000	000064